

KNAV · CVIČENÍ 1

Jak vytvářet objekty

Factory Method · Abstract Factory · Prototype · Builder · Singleton

Pět vzorů. U každého jedna otázka, jeden příklad a kousek kódu.

PŘEHLED

Co přesně se snažíme zjednodušit?

Vzor	Hlavní otázka
Factory Method	Jak vytvořit objekt a nesvázat kód s konkrétní třídou?
Abstract Factory	Jak vytvořit celou sadu souvisejících objektů?
Prototype	Jak rychle vytvořit kopii existujícího objektu?
Builder	Jak čitelně sestavit jeden složitý objekt?
Singleton	Jak zajistit jednu sdílenou instanci?

FACTORY METHOD

„Nech podtřídu rozhodnout, co se vytvoří“

POUŽIJ, KDYŽ

kód potřebuje objekt, ale nemá se vázat na jeho konkrétní třídu.

PŘÍKLAD

aplikace posílá notifikace. Zda půjde e-mail, nebo SMS, rozhoduje podtřída přes tovární metodu.

```
abstract class NotificationService {  
    abstract Sender createSender();  
    void notify(String msg) {  
        createSender().send(msg);  
    }  
}  
  
class EmailService extends NotificationService {  
    Sender createSender() { return new EmailSender(); }  
}
```

JEDNOU VĚTOU Factory Method nechává podtřídy rozhodnout, jakou konkrétní třídu vytvoří.

ABSTRACT FACTORY

„Vytvoř celou sadu souvisejících objektů“

POUŽIJ, KDYŽ

potřebuješ víc objektů, které k sobě patří a mají být ze stejné rodiny.

PŘÍKLAD

světlý a tmavý vzhled aplikace. Klient dostane DarkUiFactory a nemůže omylem spojit světlé tlačítko s tmavým dialogem.

```
interface UiFactory {  
    Button createButton();  
    Dialog createDialog();  
}  
  
class DarkUiFactory implements UiFactory {  
    public Button createButton() { return new DarkButton(); }  
    public Dialog createDialog() { return new DarkDialog(); }  
}  
  
// LightUiFactory vypadá stejně, jen se světlými třídami
```

JEDNOU VĚTOU Abstract Factory vytváří rodinu souvisejících objektů, které mají být použité společně.

PROTOTYPE

„Vytvoř kopii hotového objektu“

POUŽIJ, KDYŽ

je vytvoření objektu náročné, nebo máš hotový objekt, který chceš opakovaně kopírovat a upravovat.

PŘÍKLAD

hra má přednastaveného nepřítele. Vytvoří se kopie a změní se třeba pozice.

POZOR

sdílí kopie vnořené objekty s originálem, nebo je kopíruje taky? To je rozdíl mezi mělkým a hlubokým kopírováním.

```
Enemy original = new Enemy("robot", 100, "laser");  
Enemy another = original.copy();  
another.setPosition(10, 20);
```

JEDNOU VĚTOU Prototype vytváří nové objekty kopírováním existujících.

BUILDER

„Sestav složitý objekt krok za krokem“

POUŽIJ, KDYŽ

má objekt hodně volitelných parametrů nebo se špatně čte jeho konstruktor.

PŘÍKLAD

objednávka burgeru: povinný základ, volitelně sýr, omáčka a příloha.

BEZ BUILDERU

```
new Burger("hovězí", true, "BBQ", true, false, ...)
```

S BUILDEREM

```
Burger burger = new Burger.Builder("hovězí")
    .cheese(true)
    .sauce("BBQ")
    .fries(true)
    .build();
```

JEDNOU VĚTOU Builder pomáhá vytvořit jeden složitější objekt čitelným způsobem.

SINGLETON

„V celé aplikaci má být jedna sdílená instance“

POUŽIJ JEN, KDYŽ

má opravdu existovat jediná instance služby a všichni k ní potřebují přístup.

PŘÍKLAD

konfigurace načtená při startu aplikace.

POZOR

snadno se nadužívá. Skrytě sdílený stav ztěžuje testování. Často stačí službu vytvořit jednou při startu a předat ji těm, kdo ji potřebují.

```
enum AppConfig {  
    INSTANCE;  
  
    public String getMode() {  
        return "production";  
    }  
}  
  
// použití:  
String mode = AppConfig.INSTANCE.getMode();
```

JEDNOU VĚTOU Singleton omezuje vytvoření instance na jednu sdílenou instanci.

SHRNUTÍ

Pro rychlé zapamatování

Vzor	Jednou větou
Factory Method	Nechá podtřídy rozhodnout, jakou konkrétní třídu vytvoří.
Abstract Factory	Vytváří rodinu souvisejících objektů, které mají být použité společně.
Prototype	Vytváří nové objekty kopírováním existujících.
Builder	Pomáhá vytvořit jeden složitější objekt čitelným způsobem.
Singleton	Omezuje vytvoření instance na jednu sdílenou instanci.

A TEĎ DO KÓDU

Podíváme se, jak to vypadá doopravdy

Spustíte Exercise1PoC a podívejte se na výstup.

Pak otevřeme balíček každého vzoru a projdeme ho společně.

Zkuste něco změnit: další variantu, jinou hodnotu, jiné nastavení.

```
src/exercises/exercise1/  
├─ Exercise1PoC.java  
└─ patterns/  
    ├─ factorymethod/  
    ├─ abstractfactory/  
    ├─ prototype/  
    ├─ builder/  
    └─ singleton/
```

github.com/JakubObornik/KNAVProject